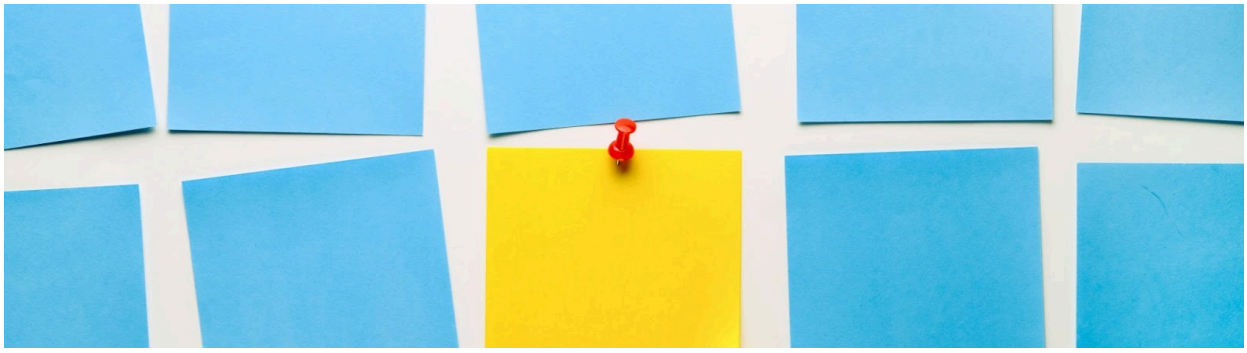

Git Hub Link :- https://github.com/Diyajain1803/Product-Store-Listing

Project deployment Link :-

https://product-store-listing.vercel.app/

📘 PRODUCT LISTING WEB APPLICATION

Technical Documentation

---

1. Abstract

The **Product Listing Web Application** is a full-stack web solution designed to efficiently manage and display products with dynamic filtering and sorting capabilities. The system enables users to browse products, apply multiple filters (category and price range), sort data, and add new products with optional image uploads.

The application follows a **client-server architecture**, where the frontend communicates with a backend API built using Node.js and Express. All filtering and sorting operations are handled on the server side to ensure performance optimization and scalability.

---

2. Introduction

Modern e-commerce platforms require efficient product management systems that provide users with seamless browsing experiences. This project aims to replicate core functionalities of such systems, including:

- Dynamic product listing
- Multi-criteria filtering
- Sorting mechanisms
- Product addition with validation

The application demonstrates practical implementation of **RESTful APIs**, **frontend-backend integration**, and **data handling techniques**.

---

3. System Architecture

3.1 Architecture Type

The system is based on a **3-tier architecture**:

Presentation Layer (Frontend)
↓
Application Layer (Backend API)
↓
Data Layer (In-Memory Storage)

3.2 Workflow

1. User interacts with UI (filters, sorting, add product)

2. Frontend sends HTTP requests to backend
3. Backend processes request (filter/sort/validate)
4. JSON response returned
5. UI dynamically updates

---

4. Technology Stack

4.1 Backend Technologies

- **Node.js** – JavaScript runtime environment
- **Express.js** – Lightweight web framework
- **CORS Middleware** – Enables cross-origin communication

4.2 Frontend Technologies

- **HTML5** – Structure of web pages
- **CSS3** – Styling and layout
- **JavaScript (Vanilla)** – Client-side logic

4.3 Supporting Tools

- **Base64 Encoding** – Image handling
- **LoremFlickr API** – Sample images

- **Nodemon (Dev Tool)** – Auto-restart server

---

5. Project Structure

```
product-store/
├── server.js            # Backend server logic
├── package.json         # Dependencies & scripts
├── package-lock.json    # Dependency lock file
└── public/
  ├── index.html       # User Interface
  ├── style.css        # Styling
  └── app.js           # Frontend logic
```

---

6. Backend Design

6.1 Server Configuration

The backend is implemented using Express.js with the following middleware:

- JSON Parser → Handles request body
- CORS → Enables frontend-backend communication
- Static Middleware → Serves frontend files

6.2 Data Storage

- Uses **in-memory array**
- No database integration

- Data resets on server restart

6.3 API Endpoints

6.3.1 Get Categories

Endpoint: `GET /api/categories`
Purpose: Fetch unique product categories

6.3.2 Get Products

Endpoint: `GET /api/products`

Query Parameters:

- `categories` → Filter by category
- `minPrice` → Minimum price
- `maxPrice` → Maximum price
- `sort` → price_asc / price_desc

Processing Steps:

1. Clone product list
2. Apply category filter
3. Apply price filter
4. Apply sorting

5. Return filtered result

6.3.3 Add Product

Endpoint: `POST /api/products`

Input Fields:

- Name
- Price
- Category
- Image (optional)

Validation Rules:

- Name must not be empty
- Price must be numeric and positive
- Category must be provided

---

7. Frontend Design

7.1 HTML Structure (Tags Used)

Structural Tags

- `<html>`, `<head>`, `<body>`

Layout Tags

- `<div>`, `<section>`, `<header>`

Form Elements

- `<form>`
- `<input>` (text, number, file)
- `<select>` and `<option>`
- `<button>`

Content Elements

- `<h1>`, `<h2>`
- `<p>`, `<span>`
- `<img>`

7.2 CSS Design

- Grid and Flexbox for layout
- Card-based UI design
- Responsive interface
- Custom font integration

7.3 JavaScript Functionality

- Fetch API for HTTP requests
- Event listeners for user interaction
- Dynamic DOM rendering
- Form validation
- Image preview using FileReader

---

8. Core Functionalities

8.1 Product Listing

- Displays products in card format
- Shows image, name, price, category

8.2 Filtering System

- Multi-category selection
- Price range filtering
- Real-time updates

8.3 Sorting Mechanism

- Ascending (low → high)
- Descending (high → low)

8.4 Add Product Feature

- Input form with validation
- Image upload support
- Immediate UI update

---

9. Image Handling

- Images are converted to **Base64 format**
- Sent via JSON request
- Stored in memory
- Preview displayed before upload

---

10. Performance Optimization

- Filtering handled on backend (not frontend)
- Reduces client-side computation
- Improves scalability for large datasets

11. Limitations

- No persistent database
- Data loss on server restart
- No authentication/authorization
- Image storage not optimized

12. Future Enhancements

- Database integration (MongoDB / MySQL)
- User authentication system
- Pagination and search functionality
- Cloud-based image storage (AWS S3 / Cloudinary)
- Advanced UI/UX improvements

13. Testing & Validation

- Input validation implemented for forms
- API responses verified using browser tools

- Edge cases handled (invalid inputs, empty fields)

---

14. Conclusion

The Product Listing Web Application successfully demonstrates the implementation of a scalable and efficient product management system. It highlights key software engineering concepts such as:

- RESTful API design
- Client-server communication
- Dynamic UI rendering
- Data validation and handling

This project serves as a strong foundation for developing real-world e-commerce applications.